Composer: Latency and Criticality Analyses for Component-Level Programming

Abstract—**Real-time embedded systems must meet strict non-functional guarantees such as timing predictability, isolation, robustness, and resource efficiency. Meeting these guarantees requires understanding the system's underlying non-functional properties – the measurable aspects of how software units communicate, share resources, and influence each other. Because these properties arise from interactions and resource sharing that are not explicitly represented in monolithic systems, they are difficult to observe, analyze, or control. In contrast, component-based systems make these relationships explicit by decomposing the system into isolated components with interfaces and resource boundaries. However, visibility alone is insufficient – we still lack a systematic way to reason about how component interactions affect timing, interference, and other non-functional behavior across the whole system. We therefore introduce Composer, a build-time framework that orchestrates the specification and creation of a component-based OS, and analyzes and structures system composition around non-functional properties. By interpreting the composition of components – their dependencies, resource usage, and interactions – Composer derives a system-level model that captures and relates these non-functional properties, supporting the design of predictable, analyzable, and resource-efficient real-time systems. Our evaluation demonstrates that Composer enables systematic reasoning about non-functional behavior in mixed-criticality systems, and improves overall predictability, isolation, robustness and resource efficiency in the Composite component-based OS.**

I. Introduction

Embedded systems often present a highly-constrained design space for software. Size, Weight, Power, and Cost (SWaP-C) constraints lead to a co-design between the physical system, the computational infrastructure, and the system software. The system software and computational capabilities must ultimately accommodate the timing and functional requirements of the physical system. The large variety of RTOSes and middleware cater to different niches: from minimal RTOSes such as FreeRTOS [**?**] that focus on scaling down effectively to restricted hardware, and ROS-based [**?**] systems that interface with complex sensors and co-processors, up to full network-facing Linux systems that aim to provide feature-rich graphical environments.

Component-based systems span this gap, and enable systems software to scale across different embedded and real-time contexts. They enable developers to choose the set of functional components that constitute the executable system. A component is a set of *code and data* that defines the component's functionality, the interfaces it *exports* for use by other components, and it's *dependencies* on interfaces it requires from other components. These interfaces are often sets of functions (*e.g.* in Composite [**?**] or CAMKES [**?**]), or publisher-subscriber specifications (*e.g.* in ROS). The general coupling of components based on functional interfaces enables a system to include only a *minimal set* of components required for the embedded system, and enables the use of *specialized components* that provide functionality specific to the embedded system's requirements (*e.g.* resource management policies).

While these systems specialize the functional makeup of a system, an outstanding challenge of component-based systems is to understand the *non-functional and timing properties* of a given set of components. For example, in ROS 2 [**?**], executors [**?**] execute component code with run-to-completion semantics, optionally in separate processes from other components, and multiple executors can execute a component's code. A component that executes blocking calls or IPC delays all of an executor's tasks, and the end-to-end latency across a component chain is dependent on each component. Component-based OSes (CBOS) [**?**], [**?**], [**?**], [**?**], [**?**], [**?**], [**?**] define the very OS functionality itself with components. These systems not only define core system functionality like filesystems, device drivers, and networking in components, but must also carefully define the concurrency and timing properties of applications with components. This requires the intentional design and usage of kernel facilities for priority, budget, and resource sharing [**?**], [**?**], [**?**], [**?**].

In a CBOS, *system-level* components (or "system components") are those that provide their functionality to multiple applications. These often use the underlying isolation facilities of the system – for example, page-tables – to enforce strong separation between components. Understanding the end-to-end execution times of applications which invoke the functionality in system components can pose a significant challenge. A system component's timing depends on the usage patterns of *all* applications that invoke it. Mixed Criticality Systems (MCS) [**?**] – in which some applications handle tasks that are more critical than others – amplify this challenge. When applications of different criticality levels depend on shared system components, cross-criticality interference can manifest in delays in high-criticality tasks. Lower-criticality applications often are of lower assurance – subject to less complete testing and validation – as they tend to be much more complex. As such, high-criticality tasks can suffer latency degradation through system components due to low-assurance tasks. Composer is motivated by a key fact: global properties (*i.e.* end-to-end high-criticality task latencies) are heavily impacted by local factors – how specific components are related to each other, and the properties of those components.

Worse, in a CBOS, the extensibility of the system means that below multiple layers of component dependencies, system components impacted by low-assurance tasks can threaten high-criticality execution predictability. Composer aims to ensure that global system properties are not threatened by chosen system components using a few core techniques. System components define their own "virtual resources" – for example, a scheduler component provides threads, memory manager component provides shared memory regions, and an IPC component provides channels – and how these resources are utilized and shared between other components has a direct

impact on isolation. Composer takes a specification of each system component's resources, and how they are allocated to and used by other components (of varying criticalities). Further, system components that use critical sections to protect their implementation can induce interference on clients, especially when triggered from low-assurance components with unpredictable usage patterns. In each case, the system detects if low-criticality components can induce undue (and potentially unpredictable) delays in high-criticality components, and introduces a set of *component-graph transformations* enable the decoupling of the timing properties across criticalities. Composer additionally optimizes components given the knowledge of the full component graph by enabling them to pre-allocate appropriate amounts of resources, and to ensure that "client" components have appropriately restricted access.

We build Composer on top of the Composite component-based OS [**?**] which is a μ-kernel [**?**]. Composite is an open source system that spans from micro-controllers with 100s of KB of memory [**?**], [**?**] up to cloud systems handling Gb/s of client requests [**?**], [**?**], [**?**], [**?**]. As a μ-kernel, device drivers, filesystems, network stacks, and similar system policies are defined in components. However, in Composite, even otherwise low-level systems policies such as memory management, scheduling, and synchronization are component-defined at user-level. Composite requires Composer to scale to complex systems with many interlocked components. As all system policy is defined in components, it is important that a complex composition of components has both tightly constrained access rights and properties that enable mixed-criticality isolation.

Composer's contributions include:

- Composer enables the component-definition of abstract *virtual resources*, and captures their usage and sharing relationships in the component-based system specification. This enables the intelligent static allocation of virtual resources, facilities to avoid DoS attacks on resource allocation between criticalities, and their capability-defined access rights. We further map this specification down to the statically required kernel resources to enable each component, and transparently orchestrate their allocation.
- Composer provides powerful static analysis over the component graph to optimize components, check access control rights, and assess potential inter-criticality interference that can threaten latency properties of high-criticality computations, and the ability for a fault in a low-criticality component spreading to a high-criticality one. We introduce two component graph transformations to address interference found in the analysis.
- We implement Composer as a build time intelligent linker, and a *constructor* component to load the entire CBOS and orchestrate properly access-controlled resource allocation in Composite.
- We evaluate the ability of the Composer to enable component-level programming and analysis. When we identify timing and inter-criticality issues, we define novel ways to solve them by programming at the component-level, and optimize resource allocations to the requirements of the component graph.

II. High-Level Design of Composer

Composer enables developers to program at the component level rather than configuring monolithic programs and systems. Developers produce a component-system specification that declares the structure, dependencies, and non-functional relationships among components. Composer analyzes this specification and generates a bootable component-based system that satisfies – and can further optimize toward – the key goals illustrated in Figure **??**. This section presents an overview of Composer's architecture and workflow, followed by an example showing how Composer supports component-level programming.

A. Overview of Composer

Composer consists of a set of system and build abstractions that take a system specification, analyze it, and generate a bootable system, as shown in Figure **??**.

The system specification defines each component's interfaces, dependencies, required resources. In addition to the textual specification, binary artifacts serve as a separate input source that provides information about synchronous invocation between components. More details of the specification are presented in Section **??**.

Composer performs the analyses shown in Figure **??** to derive the system's functional and non-functional properties. The analyses focus on reasoning about resource allocation and capturing criticality-aware interactions across the entire system. By combining these analyses, Composer gains a deep understanding of how components interact. Thus, Composer can, in mixed-criticality systems, trace how low-criticality components affect high-criticality ones by identifying interference paths through shared resources, dependencies, and synchronization channels. All of these analyses are described in detail in Section **??**.

The output of Composer is a bootable system image accompanied by a set of configuration artifacts. These artifacts encode the resource allocations, dependencies, and access rights derived during analysis. Guided by these configurations, the runtime environment ensures during boot-up that each component is allocated only its designated resources and has only the minimal access necessary to those resources. More details of the runtime composition process are presented in Section **??**.

These three phases together enable two features of Composer: resource allocation optimization and mixed-criticality isolation and interference analysis. These features are fundamental to how Composer supports component-level programming and how it analyzes the latency and criticality of the generated component-based system.

Resource allocation optimization. Typical resources include threads, page tables, capabilities, files, sockets, communication channels, and shared memory. The resources in the input specification describe not only the resources themselves (e.g., their size, quantity, and ownership) but also the interfaces used to access those resources and the dependencies that specify which components provide and which components consume each resource. The resource allocation and analysis determine

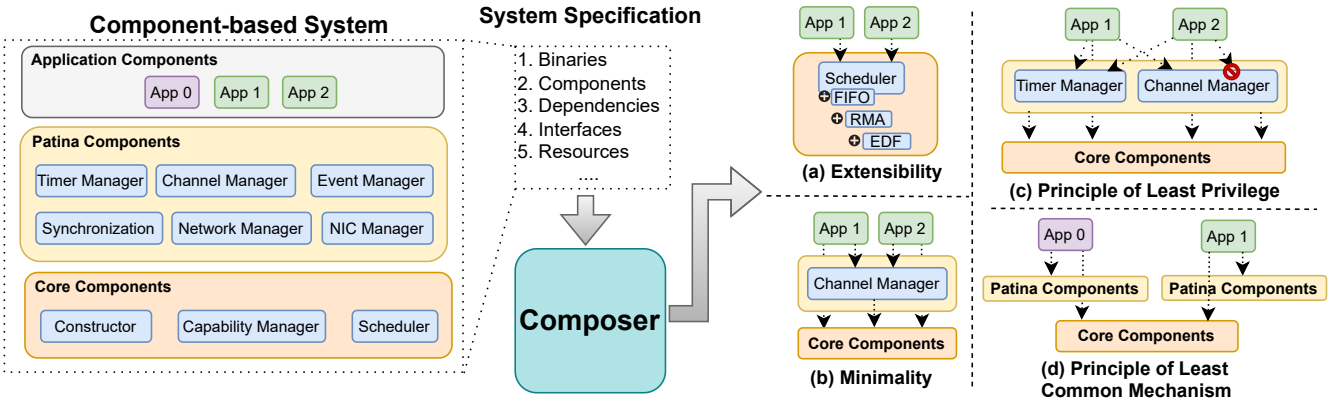


Fig. 1: This figure illustrates Composer's core design goals for component-based systems. The left side of the figure shows a component-based operating system architecture using Patina as an example. Patina is an RTOS-API layer designed for microkernel systems that adheres to the principle of least privilege and provides common OS services across components. The four subfigures on the right—(a) through (d)—demonstrate how Composer applies these goals in different scenarios. (a) Extensibility: different implementations of the scheduler component (e.g., FIFO, RMA, EDF) can be substituted without modifying the client components (App1 and App2). This demonstrates component-level polymorphism, where multiple policies satisfy the same scheduler interface. (b) Minimality: unnecessary components can be omitted. In this example, App1 and App2 depend only on the channel manager, so all other patina components are removed from the final system configuration. This reduces resource footprint and minimizes attack surface. (c) Principle of Least Privilege (PoLP): each component is granted only the capabilities required for its permitted operations. Here, App2 does not hold a capability to access the channel manager's protected operations; this restriction prevents unauthorized access to shared resources. (d) Principle of Least Common Mechanism (PoLCM): In this example, App0 and App1 have different criticality levels, and each application is connected to its own patina components. By avoiding transitive dependencies on a common mechanism across criticalities, this configuration effectively minimizes cross-criticality interference.

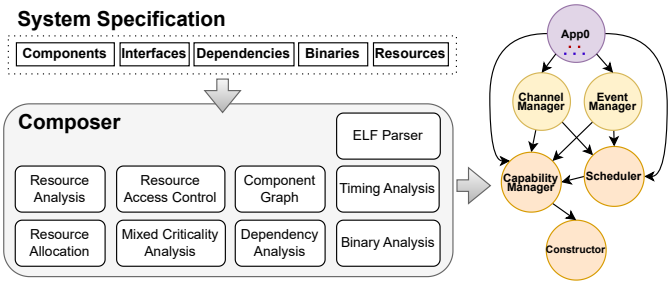


Fig. 2: Composer workflow: illustrates how Composer operates within a component-based system across three stages — component system specification, analysis for component-level programming, and runtime composition of components.

```
1: [[components]]
2: component = "chan_hc"
3: criticality_level = 1
4: virt_res = [{vr_type = "chan", instance = "chan1"}]
5: deps = [{srv = "chanmgr"}]
6: [[components]]
7: component = "chan_lc"
8: criticality_level = 0
9: virt_res = [{vr_type = "chan", instance = "chan1"}]
10: deps = [{srv = "chanmgr"} , ... ]
11: [[virt_resources]]
12: vr_type = "chan"
13: component = "chanmgr"
14: resources = [{instance = "chan1", param = {slots =
"6"}}]
```

Fig. 3: Simplified example of a Composer input specification showing two components of different criticality levels sharing a virtual resource.

which components own or manage each resource and which components rely on them. Together with resource access control, this reasoning enables Composer to resolve resource allocations with capabilities, ensuring that each resource is correctly assigned to its designated components and that only components with the appropriate access rights can use it. At runtime, the system consumes this allocation information: during boot-up, the runtime environment follows the Composer-generated configuration to allocate the specified resources to each component and enforce the derived access rights.

Mixed-Criticality isolation and interference analysis. The input specification provides the criticality level of each component, indicating its assurance requirements and the potential system impact of its failure. The mixed-criticality analysis within the analysis traces how components of different criticality levels connect through the dependency graph. Consider a simple case: one high-criticality component and one low-criticality component both depend on a shared resource provided by a third component. If the low-criticality component only reads from this resource, the interaction is considered safe, as its operations cannot affect the high-criticality component's behavior. However, if both high- and low-criticality components can modify the shared resource, the analysis flags potential interference that could violate high-criticality constraints. Beyond such simple cases, this reasoning extends across the entire directed acyclic graph (DAG) of components. For example, when a low-criticality component indirectly influences a high-criticality one through several intermediate components, Composer identifies these indirect dependencies as potential interference paths.

Component-level programming with example. Figure **??** presents a simplified example of the system specification format. Each component entry defines its identity, `criticality_level`, and the virtual resources it requires. In this example, two components – `chan_hc` and `chan_lc` – represent high- and low-criticality tasks, respectively. Both specify `deps = [{srv = "chanmgr"}]` to indicate that they depend on the same service component `chanmgr`, which provides the shared virtual resource `chan1`. This input struc-

ture allows Composer to infer how resources are shared and to reason about access rights, ownership, and potential interference across components.

III. System Model

A. Components, Dependency, Interface, Function

Let $C = \{c^x, \ldots\}$ denote the set of system components, where x is an index identifying a component.

For each component $c^x \in C$, let $d^x \subseteq C$ denote direct dependencies of c^x; that is, $c^y \in d^x$ if c^x uses functionality provided by c^y via c^y's interface. The transitive closure of c^x is $D^x = d^x \cup \bigcup_{c^y \in d^x} D^y$.

Interface attributes E^x denote the set of interfaces exported by c^x. U^x denote the set of interfaces used by c^x.

We say that (c^x, c^y) is a provider–client pair if c^y uses at least one interface exported by c^x. For each provider–client pair (c^x, c^y), we define the interface binding set

$$I^{x \to y} \triangleq E^x \cap U^y.$$

For each provider–client pair (c^x, c^y) and each interface instance $i_n \in I^{x \to y}$, let $\overleftarrow{F}^x_{i_n}$ denote the set of functions provided by component c^x via interface i_n, and let $\overrightarrow{F}^y_{i_n}$ denote the set of functions used by component c^y via interface i_n. We define the least-privilege function set

$$F^{x \to y}_{i_n} \triangleq \overleftarrow{F}^x_{i_n} \cap \overrightarrow{F}^y_{i_n}.$$

B. Component Attributes

Virtual resource Let $\overleftarrow{R}^x$ denote the set of virtual resource instances that component c^x provides. Let $\overrightarrow{R}^x$ denote the set of virtual resource instances that component c^x uses.

Virtual resource access Let A_{ac} denote the set of atomic access rights:

$$A_{\text{ac}} = \{\mathsf{read}, \mathsf{write}, \mathsf{blocking}, \mathsf{dynamic_allocation}\}.$$

For each instance $r_n \in \overrightarrow{R}^x$, we define its access control rights for component c^x as $\mathsf{ac}(c^x, r_n) \subseteq A_{\text{ac}}$.

Criticality level component may have a level of criticality L. We assume that tasks from a component c^x with L^x have higher priority than tasks from component c^y with L^y whenever $L^x > L^y$.

Lock attribute We define a boolean lock attribute $\lambda^x \in \{false, true\}$, where $\lambda^x = true$ indicates that component c^x may hold a lock and $\lambda^x = false$ indicates no such lock usage.

C. Virtual Resource Allocation and Coordination

Provider–client binding. We say that component c^x uses instance r_n from component c^y if

$$c^y \in D^x \ \wedge \ r_n \in \overrightarrow{R}^x \cap \overleftarrow{R}^y.$$

Accordingly, we define the provider–client binding relation

$$B^{x \to y} = \left\{ (c^x, r_n, c^y) \;\middle|\; c^y \in D^x \ \wedge \ r_n \in \overrightarrow{R}^x \cap \overleftarrow{R}^y \right\}.$$

Resource allocation. When a component allocates a resource instance r_n, it requires allocation parameters that determine how the instance is created.

Let K_{r_n} denote the set of parameter names (e.g., size, priority, number, ...) for r_n, and let V_{r_n} denote the set of parameter values (e.g., integers, booleans, strings) for r_n. We write $\mathsf{param}(r_n, k_n, v_n)$, where $k_n \in K_{r_n}$ and $v_n \in V_{r_n}$, to denote that instance r_n is allocated with parameter $k_n = v_n$.

An instance r_n may have multiple allocation parameters; we represent them as a set of key–value pairs. For each instance r_n, define its parameter set

$$P_{r_n} = \{\, (k, v) \in K_{r_n} \times V_{r_n} \mid \mathsf{param}(r_n, k, v) \,\}.$$

For a component c^x and a resource instance $r_n \in \overleftarrow{R}^x$, we write $\mathsf{alloc}(c^x, r_n, P_{r_n})$ to denote that c^x allocates instance r_n with parameters P_{r_n}.

Resource coordination. Any resource instance used by a client must be allocated by its provider:

$$\forall c^x, c^y \in C, \forall r_n : \ (c^x, r_n, c^y) \in B^{x \to y} \ \Rightarrow \ \mathsf{alloc}(c^y, r_n, P_{r_n}).$$

Moreover, we impose an initialization order consistent with component dependencies. Write $y \prec x$ to mean that c^y is initialized before c^x, and require:

$$c^y \in D^x \ \Rightarrow \ y \prec x.$$

D. Safety Conditions and Constraints

Least-privilege refinement The dependency closure D^x determines the set of server components that are reachable by c^x, while Least-privilege refinement determines which specific functions within those reachable components' interfaces are actually invoked.

$$\mathcal{T}^x \triangleq \left\{ (c^y, i_n, f) \;\middle|\; \begin{array}{l} c^y \in D^x \ \wedge \ i_n \in I^{y \to x} \ \wedge \\ f \in F^{y \to x}_{i_n} \end{array} \right\}. \tag{1}$$

Interference risk Given the bindings $(c^x, r_n, c^y) \in B^{x \to y}$ and $(c^z, r_n, c^y) \in B^{z \to y}$, an interference risk exists whenever

$$L^x > L^z \ \wedge \ \mathsf{ac}(c^z, r_n) \nsubseteq A_{\text{ro}}.$$

Dos attack risk. Given the bindings $(c^x, r_n, c^y) \in B^{x \to y}$ and $(c^z, r_n, c^y) \in B^{z \to y}$, a DoS attack risk exists whenever

$$L^x > L^z \ \wedge \ \mathsf{dynamic_allocation} \in \mathsf{ac}(c^z, r_n).$$

Blocking risk. Given the bindings $(c^x, r_n, c^y) \in B^{x \to y}$, $(c^z, r_m, c^y) \in B^{z \to y}$, $(c^w, r_n, c^y) \in B^{w \to y}$, and $(c^w, r_m, c^v) \in B^{w \to v}$, a blocking risk exists whenever

$$\begin{aligned} L^x > L^z \ &\wedge \ \mathsf{blocking} \in \mathsf{ac}(c^w, r_n) \\ &\wedge \ \mathsf{blocking} \in \mathsf{ac}(c^w, r_m) \\ &\wedge \ \mathsf{blocking} \in \mathsf{ac}(c^x, r_n). \end{aligned}$$

Here, the providers c^y and c^v (and their interfaces) may coincide or differ; the condition does not require r_n and r_m to be provided by the same component.

Lock risk. A mixed-criticality lock risk exists if

$$\exists\, c^x, c^y, c^z \in C : \ L^x > L^y \ \wedge \ c^z \in D^x \cap D^y \ \wedge \ \lambda^z = true. \tag{2}$$

IV. Component System Specification

Composer begins system creation from a structured system specification that describes all components in the system, their interfaces, dependencies, and resource requirements. This section covers interfaces, dependencies, and resources—both kernel resources and virtual resources—and focuses on virtual resources, as they are central to reasoning about resource allocation, isolation, and interference at build time.

A. Component Interface, Dependencies

Interface. An interface represents a set of functions that a component exposes to the rest of the system. In a component-based system, the interface abstracts away the implementation; therefore, the implementation of these functions may change, be optimized, or be replaced entirely without affecting other components.

Dependencies. A dependency is declared when a component requires access to an interface exported by another component. This establishes a functional relationship—component A depends on component B—which indicates that A will invoke the functions provided by B.

Kernel resource. The system specification does not explicitly reference kernel resources. However, the kernel resources are implicitly encoded in other parts of the specification. In particular, the interfaces implemented by components and the dependencies declared between components implicitly determine the required kernel resources—the synchronous invocation capabilities that enable cross-component calls. Although the specification only describes functional relationships, these relationships directly imply the kernel-level allocation and configuration needed to support them.

B. Virtual Resources

We introduce a new abstraction called virtual resource, which represents each service that is provided by a server component and consumed by a client component. Server components, also known as managers, expose services through interfaces that include a set of functions, while client components access these services via those same interfaces. Each virtual resource is referenced through an opaque identifier. This identifier abstracts the underlying implementation details, allowing clients to utilize resources without directly handling resource allocation. Virtual resources are defined with configurable parameters that apply across manager and client components. This parameterization allows virtual resources to adapt to specific requirements of each client. Each virtual resource has an access policy that restricts functions within interface to certain clients based on permissions. Access rights are mapped to the functions exposed by the manager component, controlling whether a client can read, write, or execute specific operations.

Composer supports a variety of virtual resource types. In this paper, we focus on common virtual resource types such as threads, shared memory, channels, events, and timers, but the abstraction generalizes to many other resources (e.g., sockets, files, containers).

- **Thread** provides an execution context within a client and is managed by the scheduler.
- **Shared memory** provides a common memory space accessible to multiple clients.
- **Channel** allows a client to open, close, and transfer data according to its access rights.
- **Event** notifies clients and enables synchronization between them.
- **Timer** provides timing functionality for operations triggered by intervals or deadlines.

We categorize the resource-provisioning relationships between servers and clients, as shown in Figure **??**, to illustrate how virtual resources are allocated and how access rights regulate their use. These relationships include single-client, two-client, and multi-client provisioning, in which a single server supplies a resource to one or more clients.

V. Analyses for Component-level Programming

After consuming the system specification, Composer reasons about both functional and non-functional aspects of the system. It performs a number of analyses on the component-based system based on their dependencies, virtual resources, criticalities, and their binaries.

A. Dependency Analysis

The dependency and interface specification describes functional relationships between components: which component relies on which service, and through which exported interface that service is accessed. It then determines which functions in the interface the client calls, thus which require IPC-facilitated invocations to the server functions. During dependency analysis, Composer constructs a directed acyclic graph where vertices represent components and edges represent dependencies. This graph captures the functional topology of the system. Composer uses this graph to reason about ordering constraints, isolation boundaries, potential interference, resource allocation, and other system-wide interactions.

B. Inter-Component Analysis

Criticality propagation. Our analysis begins by generating the transitive closure of dependencies to identify the full range of interactions among components. This mapping reveals the paths through which criticality levels can influence one another, allowing us to trace how criticality propagates throughout the system. With the transitive dependencies mapped out, we calculate the criticality exposure for each component and define its criticality range. This criticality range reflects the aggregated impact of criticality levels from all dependent components on a given component. It allows us to detect criticality propagation risks, such as a low criticality component indirectly depending on a high criticality component.

Logical interference. Our analysis then proceeds by examining each component's properties and virtual resource access patterns. Properties are fundamental attributes that describe a component's intrinsic behaviors, while virtual resource access patterns define how components interact with shared resources through operations such as read, write, blocking, and dynamic allocation. With criticality propagation, component properties, and access patterns mapped, we can trace how each component's behaviors and access patterns influence information flow across the system. This enables us to detect

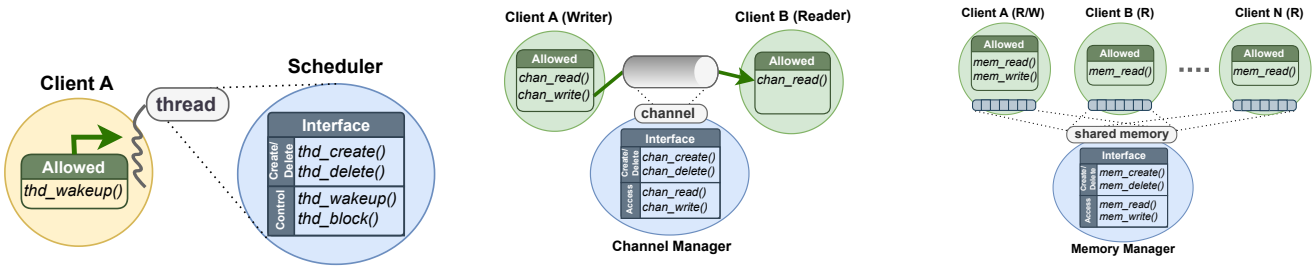


Fig. 4: This figure illustrates typical provisioning patterns, where one server-managed resource (e.g., a thread, channel, or shared memory region) is shared by one or more clients with different access rights.

potential logical interference by identifying instances where a component's properties or access patterns may affect the specific data and control interactions that high criticality components rely on to perform their logic.

Integrity analysis. We further address integrity concerns by incorporating assurance levels into our analysis. In our system, assurance levels are assigned to components to reflect the confidence in their correct operation. Core components, which are essential to system stability, are assigned the highest assurance levels, while application-level components typically have low assurance levels. Our analysis uses assurance levels to filter out warnings for trusted components, as these components, with their high assurance levels, are expected to operate without issues. This approach allows us to concentrate on warnings for components exposed to criticality levels beyond their assurance capabilities, ensuring that attention is directed toward areas with potential risks.

C. Resources Analysis

Virtual resources are shared resources that components can access based on predefined access rights and configuration. Access rights for each virtual resource may include read, write, blocking, and dynamic allocation, which control the ways components interact with these resources. The configuration specifies which components share virtual resources and how virtual resources are associated with one another. Our analysis leverages the configurations and access rights to detect potential interference or contention, particularly where high and low criticality components share the same resource. By identifying how each component's access rights affect shared resources, the analysis can pinpoint risks where low criticality components might inadvertently impact high criticality operations.

Limiting API exposure. We can further control the API exposure of a virtual resource by restricting access rights for client components. Since each function within a component has its own access permission, if a client component only has read access to a virtual resource, it is restricted to calling only functions with read permissions from the component that provides this virtual resource. If we detect risks where low criticality components could impact high criticality components through shared resources, we can restrict the ability of low criticality components to interact with these virtual resources. This prevents them from making modifications or triggering operations that might compromise the stability or performance of high criticality components.

Virtual resource threads. For virtual resource threads, we can calculate the maximum number of statically allocated threads per component using the bounded thread numbers specified for application-level components in the configuration. Because core and service components are generally infrastructure elements designed to support other components, their thread requirements adapt based on the needs of their clients. Thus, specifying bounded thread numbers is unnecessary for these components, as their thread count aligns with client demand. For core and service-level components, we assume that their threads consist of a set number of initialization threads plus any threads allocated by their client components. In our analysis, the Datalog relations and supporting logic calculate and store the total thread count for each component, providing a clear overview of each component's thread allocation needs.

D. Inter-Criticality Timing Analysis

In mixed criticality environments, ensuring that high criticality components meet strict timing requirements is essential. This analysis identifies potential timing hazards that could arise from interactions between high and low criticality components, helping us detect risks where timing guarantees might be compromised.

Potential inter-criticality interference. When a component with a lock is shared between high criticality and low criticality components, a high criticality component might be forced to wait if a low criticality component is holding the lock. This can lead to priority inversion, where the high criticality component experiences delays due to low criticality operations, increasing interference latencies and potentially impacting system performance.

Limiting potential unbounded synchronization. When a high criticality component relies on a shared virtual resource accessed by both high and low criticality components, there is a risk of unbounded latency. Shared resource contention can allow a low criticality component to directly or indirectly block a high criticality one, potentially violating critical timing requirements and introducing delays that are unpredictable for high criticality operations.

Potential DoS on higher-criticality components. If both high and low criticality components can dynamically allocate the same virtual resource, there's a risk that the low criticality component could exhaust the resource. This poses a potential DoS attack where a high criticality component might be starved of necessary resources.

E. Binary Analysis

Composer includes a number of *binary analyses* that are run on each component. Once a binary is generated for each component, these analyses aim to provide function call graphs and stack usage information for simple components. This information is used (1) to shrink the memory within the component allocated for stacks, and (2) integrate with the inter-component dependency specification to shrink the number of server functions accessible by components.

We implement the binary analysis using the Capstone library [**?**]. First, we generate a functional call graph representation to understand which functions call which others. This analysis requires constrained code that makes no dynamic function calls through pointers. Embedded system code is often written within this restriction [**?**], [**?**]. Composer enables code extensibility and specialization using components, thus most components in our practical system adhere to this guideline. Second, we generate the stack size requirements of each function with the goal of bounding stack consumption. This assumes that invocation frames are statically sized (*e.g.* no `alloca` nor variable length arrays) and that there is no recursion, both of which are common embedded system requirements. Both analyses are sound and complete given these assumptions. To calculate tight stack-size requirements for each component, we track the maximum accumulated stack size across all function call paths in the component. Composer feeds this value back into the build system to specialize the component binary to the tight stack size.

The second analysis aims to enforce the principle of least privilege by ensuring that client components are only given the ability (the capability) to invoke a specific function in a server component's interface if the binary requires that function. This refines the function access control in the system from enabling client components to access functions in interfaces associated with dependencies to additionally only allowing access to required functions. While this doesn't functionally change the system, should a component be compromised, it prevents it from exercising rights beyond what it should require. This analysis integrates the inter-component dependency analysis to understand which interfaces connect components, with the binary analysis that provides the mapping between each exported function and `main`, with the set of functions they can call in other component's interfaces. Composer iterates through all components, removing the ability to call unused functions, until it reaches a fixed point in which no changes are made.

F. Datalog Implementation

Composer utilizes Datalog to perform a criticality-aware analysis of our component-based system. Datalog is a declarative language that is a subset of Prolog, designed for expressing logical relations and recursive queries over relational data. Datalog is a very simple language based on a restricted form of first-order predicate logic, but powerful enough to do complicated graph computations. We use Datalog to provide high-confidence analyses with terse representations. We use the Ascent [**?**] Datalog engine to implement our analysis. In our analysis, Datalog models components, their properties, virtual resource accesses, dependencies, assurance levels, and criticality levels, and it generates warnings based on the analysis.

```
depends_on(c, s) <-- dependencies(c, s, _, _);
depends_on(c, ss) <-- depends_on(c, s) ∧
         dependencies(s, ss, _, _);
criticality_exposure(c, crit) <--
        criticalities(c, crit);
criticality_exposure(s, crit) <--
        depends_on(c, s) ∧ criticalities(c, crit);
```

This depicts a fragment of an analysis in Datalog. These rules should be read as "if the conjunction of the statements on the right is true, then the statement on the left of the `<--` is true". From the input `dependencies(c, s, ...)` – which denote the specification-derived inter-component dependencies from `c` to `s` – the `depends_on` relation computes the transitive closure over those dependencies. From the input annotations on which components `c` have which `criticalities`, the `criticality_exposure` computes for all components, which criticalities of computations can (through a call-chain) invoke their functionality. In this example, if a component has exposure to both high and low criticality, and it uses mutually-exclusive synchronization (not shown here), it is a candidate for inter-criticality interference. All analyses in Composer are less than 200 lines of Datalog.

VI. Runtime Composition of Components

After the system specification and analysis phases, the final stage of Composer's workflow is runtime realization, where the analyzed configuration is instantiated into a running system. In this phase, the configuration artifacts generated by Composer guide the component-based operating system in constructing and initializing its component graph. Using Composite as an example, the runtime phase demonstrates how build-time reasoning about resource allocation, dependencies, and access control is materialized within the component-based environment.

A. Runtime Composition

The runtime composition phase instantiates the results of Composer 's build-time analysis into an executable system. Using the configuration artifacts generated by Composer, the runtime environment systematically constructs the component graph, allocates both kernel and virtual resources, and initializes each component in a controlled order. The following parts describe the main steps of this process — beginning with configuration delivery, continuing through constructor-based system initialization and scheduling, and concluding with runtime allocation of virtual resources and kernel resources.

Composer configuration of runtime components. After completing the static analysis discussed in the previous section, Composer will generate configuration information for each component based on the system specification. To facilitate the transfer of configuration information from Composer to components at runtime, Composer utilizes a hierarchical key-value (KV) data structure to store configuration data and a corresponding KV interface to provide access for each component. By supporting both dictionary-style key-value pairs and nested arrays, the KV data structure enables hierarchical organization and retrieval of configuration parameters. This design allows components at runtime to access only the configuration data relevant to their operation, thereby

simplifying data access. The KV interface provided to access the data structure is read-only, ensuring that components at runtime can retrieve but not modify the configuration data. Since this configuration data remains constant throughout execution, components operate under stable conditions set at build time. This consistency in configuration reduces unexpected behavior, promoting predictable and reliable system operation at runtime.

Constructor creation of component graph. Composer precalculates all kernel data structures necessary for the component graph and stores these as configuration information accessible to the constructor component. The constructor uses a key-value (KV) interface to access these configuration information and build the entire system. The constructor begins by iterating through these configuration information to initialize each component, which includes loading binaries from a tarball, setting up address spaces, delegating physical memory, configuring capability tables, and establishing connections between components through synchronous invocations. Composer centralizes all CFI-related authority within the constructor. The constructor holds full authority only temporarily during boot-time to set up the system, leaving no CFI-related attack surface at runtime. Furthermore, Composer pre-allocates all necessary resources for each component. Each component operates independently with its allocated resources at runtime, enabling distributed authority.

Scheduling. During system setup, the initialization order computed by Composer is included in the pre-calculated KV configuration passed to the runtime. With this ordering in place, the scheduler initializes components in a sequence that guarantees each server component is fully initialized before any client that depends on its exported interface. This ordered approach enables core and essential services to be fully operational before application-level components are initialized.

Virtual resource allocation and management. To ensure predictable and bounded memory usage, the system pre-allocates pools of virtual resources during initialization based on the pre-calculated configuration data provided by Composer. Rather than relying on dynamic allocation at runtime, each component receives a pre-determined set of resource blocks. Although this pre-allocation model is less flexible than dynamic allocation, Composer mitigates this limitation by generating per-component configuration information at build time. For server components, this configuration specifies the exact size and quantity of virtual resources to allocate for client components. For client components, the configuration defines which virtual resources they are permitted to access. At runtime, each component reads its specific configuration file to obtain the relevant virtual resource information.

This setup allows server components to allocate resources precisely based on actual requirements, rather than relying on estimations, and enables client components to access only the resources they are authorized to use, ensuring controlled resource usage. Because Composer pre-defines these configurations, the code for each component remains simple, as components can directly reference their configuration without implementing complex allocation or permission-checking logic.

Kernel resource management. In the traditional Composite design, thread allocations relied on a one-size-fits-all policy. Because running out of threads can impact system stability, Composite assigned each component a large fixed number of threads, which resulted in over-allocation and unnecessary memory consumption. In contrast, Composer's analysis computes the bounded thread count required by each component and records this information in a configuration file. During initialization, each component creates only the threads it actually needs. This approach also allows components to execute their workloads end-to-end without requiring additional synchronization for concurrency, as each component's resources are allocated precisely to match its demands.

Composer instantiates synchronous invocation capabilities (PPC capabilities) derived from dependency and binary analysis. During initialization, the constructor uses the Composer-generated configuration to allocate these capabilities in each component's capability table, ensuring that only the server functions reachable through declared interfaces are callable. Components receive no additional invocation capabilities beyond those identified by analysis, enforcing a strict principle of least privilege.

VII. Evaluation

A. Implementation and Experimental Setup

We implemented our Composer in Rust, using `Ascent` for Datalog-style logic programming, the `petgraph` library to draw the dependency graph, and a pass data structure– similar to compiler passes– that enables flexible addition and removal of features within the Composer. Our experiments utilize the Cincoze DX-1200 embedded computer as the test platform.

Cincoze's DX1200 embedded computer. The DX-1200 is a high-performance, compact, and rugged embedded computer equipped with an Intel 12th Gen Alder Lake-S Core i9-12900TE processor. This processor features 16 cores (8 performance cores and 8 efficiency cores), with clock speeds ranging from 1.1 GHz to 0.8 GHz. The system includes a 48KB L1 data cache, 32KB L1 instruction cache, 1.3MB L2 cache, and 30MB L3 cache, along with 8GB of DDR5 memory. For our evaluation, we enabled a single performance core through the BIOS, as only one core was required, and disabled hyper-threading to simplify the testing process. For result output, we use the built-in UART.

Software setup. The software environment for evaluation includes:

- Composite: Built using GCC version 11.4.0.
- Composer: Built with Rust compiler version 1.74.1.

Components. In our evaluation, the core components of Composite —including the constructor (`llbooter`), capability manager (`capmgr`), and scheduler (`sched`)—are always included as fundamental elements. Patina APIs, such as the event manager (`evt`), network manager (`netmgr`), nic manager (`nicmgr`), time manager (`tmrmgr`) and channel manager (`chanmgr`), are optionally selected based on the specific evaluation needs.

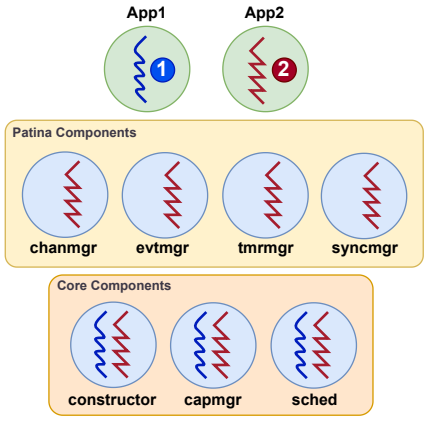


Fig. 5: This figure illustrates the impact of thread migration on the total thread count. There are two application components: App 1, which depends on core components, and App 2, which depends on both core and Patina components. With thread migration, each thread created in an application replicates into all components it depends on—For instance, a single thread created in App 1 triggers thread allocation in core components, shown by the blue thread. Similarly, a single thread created in App 2 leads to thread allocations in both Patina and core components, shown by the red thread.

B. Minimal Thread and Stack Resource Allocation

To evaluate Composer's ability to optimize resource allocation, we conducted experiments focusing on both thread number and stack size minimization. We selected thread number and stack size as our test examples because they are critical factors in resource-constrained environments, particularly when a thread migration mechanism is employed. Figure **??** illustrates the impact of thread migration on system-wide thread allocation. The baseline for these experiments is the traditional approach without any optimization.

Thread number minimization. We minimize the number of threads allocated to each component based on Composer's analysis. The stack size per thread is 8 KB by default. Figure **??** shows the stack size savings. The horizontal axis represents the incremental number of threads in a single application-level component, while the vertical axis shows the total stack size, normalized to the total stack size of the traditional Composite system.

When the number of threads in the application-level component increases, the total stack size also increases due to thread migration to dependent components. The increase is more pronounced when the application depends on all core and Patina components (green line), and less so when dependencies are limited to core components (blue line). The shaded area helps visualize the impact of partial dependencies on the total stack size. The red line represents the baseline total stack size of the traditional Composite system, where the estimated maximum number of threads is naively allocated to each component without considering minimization.

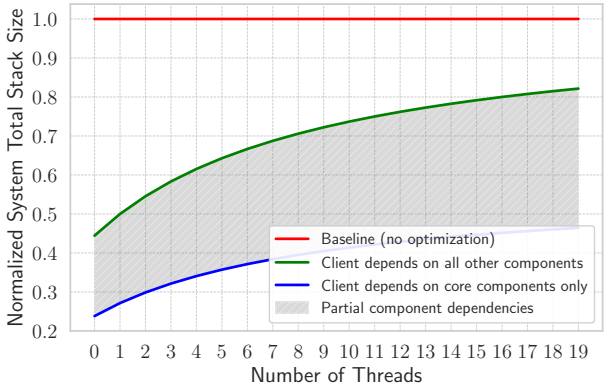


Fig. 6: Comparison of statically allocated stack sizes in baseline vs. optimized systems, highlighting the impact of increasing the number of threads in a single application-level component.

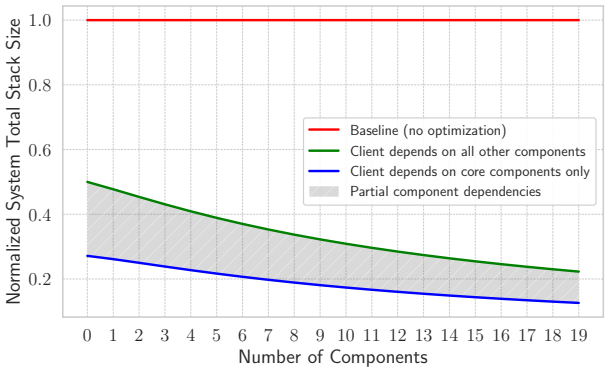


Fig. 7: Comparison of statically allocated stack sizes in baseline vs. optimized systems, highlighting the impact of increasing the number of application-level components.

Figure **??** is similar to Figure **??** but focuses on the effect of increasing the number of components while maintaining a single thread per component. The horizontal axis represents the incremental number of application-level components that have a single thread, while the vertical axis shows the same as the vertical axis in Figure **??**.

Discussion. Our thread-minimization approach (blue line, green line, the shaded area) consistently results in a lower total stack size compared to the baseline (red line). This demonstrates that our Composer framework effectively reduces unnecessary stack allocations by minimizing thread numbers.

Stack size minimization. We further adjust the stack size of each thread to match the maximum stack size required by its respective component, as determined by the Composer's analysis. The following table shows the maximum stack size for a single thread within each component.

Component	Constructor	Capmgr	Sched	Chanmgr	Evtmgr	Tmrmgr	App1	App2
Max Stack Size	4096	4096	1024	1024	1024	1024	512	512

Discussion. While the default stack size per thread is 8 KB, the Composer calculates optimized stack sizes for each thread, leading to further reductions in total stack size. Notably, the core components contain more than half of the system's threads. By customizing the stack sizes for these threads according to their actual usage requirements, we achieved more than 50% additional savings in total stack size compared to thread minimization only. This is especially beneficial in resource-constrained environments, where every KB of memory savings matters.

C. Virtual Resource Management and Coordination

To demonstrate the Composer's ability to manage component-provided abstract resources at the system level and to coordinate resource management across components,

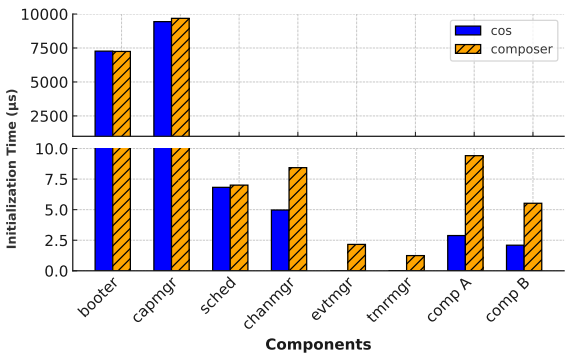


Fig. 8: Components Initialization Time.

we conducted an experiment focusing on component initialization—a key aspect of coordination and management in our framework. In our test scenario, we utilized virtual resources provided by the Patina API as a representative example. We iterated the test 10 times, as preliminary runs indicated that 10 iterations provided consistent results.

Figure **??** presents the initialization times for all components, comparing systems with and without virtual resource management and coordination. While initialization times for core components are similar, for service-level and application-level components, the initialization times are several microseconds slower than in the traditional system.

Discussion. This delay is due to two main factors: the added initialization phase for each virtual resource and the overhead introduced by our current implementation using stream read operations. The added initialization phase for each virtual resource contributes to the slight delay observed in service-level components. The use of stream read operations introduces overhead that affects both service-level and application-level components. Although the several-microsecond increment in the initialization phase is trivial, we anticipate that by replacing the current stream read operations with a more lightweight mechanism, the initialization times will be similar to those in the traditional system. This is because, despite introducing the cost of read operations, we have shifted the virtual resource initialization into the service-level components, which eliminates the IPC request overhead associated with virtual resource initialization.

D. Criticality-aware Analysis

This evaluation demonstrates Composer's criticality-aware analysis capabilities, highlighting its ability to identify and mitigate system vulnerabilities—specifically, Denial of Service (DoS) attacks, interference, and blocking issues. By generating targeted warnings and enabling straightforward modifications through system specifications, Composer provides simple solutions that enhance system stability and responsiveness for high-criticality components.

Interference warning and solution. Interference occurs when operations by low criticality components delay high-criticality components due to shared access to locked resources. This problem is inherent to any component that holds a lock across multiple criticalities, creating potential delays that could impact high criticality responsiveness.

We take the network manager using the lwIP(lightweight IP) library for packet transmission as an example. lwIP, which

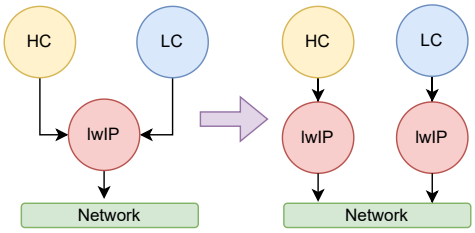


Fig. 9: Interference Mitigation through Duplication: A Composition Technique

is designed with a locking mechanism to manage access to network resources, restricts packet transmission to one component at a time. When both low and high criticality components call the shared network manager for packet transmission, the low criticality component can lock lwIP, causing the high criticality component to wait until the lock is released.

To understand the impact of interference caused by this lock on system responsiveness, we measured its effect on high criticality operations. We transmitted raw image data packets in repeated iterations (1000 times) to achieve stable interference measurements. The high criticality component's maximum packet transmission time reached 5247 CPU cycles, while the maximum interference time imposed by the low criticality component—measured as the duration of the lock—reached 10,848 CPU cycles.

In component-based systems, each component operates as an independent module with defined interfaces. This modularity enables us to address issues such as interference or resource monopolization through composition techniques. To mitigate interference in this scenario, we applied a duplication solution by introducing separate network managers for low and high criticality components, thereby isolating their access to packet transmission resources. As shown in Figure **??**, using duplication eliminated interference, allowing each component to operate independently.

Discussion. The interference time was twice as long as the original execution time, and it could become worse if the low criticality component holds the lock on the virtual resource longer. By introducing duplication, we can prevent such interference, as each component operates independently without sharing locks. Applying the composition technique is straightforward, requiring only a few lines of specification to define the duplicated component in the system configuration—no source code modifications are needed. Although duplication requires additional memory costs due to the creation of an extra component, this trade-off is necessary to ensure that high criticality components are not delayed by lower criticality operations.

Blocking warnings and solution. Blocking occurs when components rely on each other for communication but are held up by synchronous or blocking operations. In a mixed criticality system, due to blocking issue, a low criticality component can introduce unbounded latency to high criticality components.

As an example, consider the channel virtual resource that, by default, uses blocking for both send and read operations. This means that if the channel is full, any component attempting to send data will be forced to wait until a slot

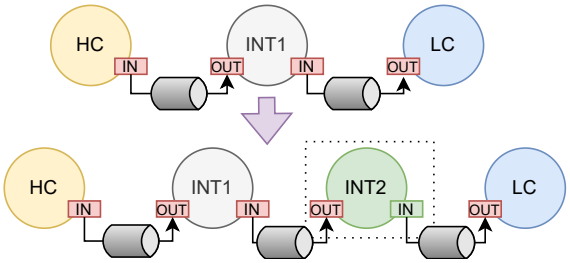


Fig. 10: This figure illustrates the use of interposition to resolve blocking issues in a mixed-criticality system. Red boxes represent blocking send and read operations, while green boxes indicate non-blocking operations. INT denotes intermediate components. Interposition inserts INT2 to manage the data flow and use a non-blocking send, preventing INT1—and thus the HC component—from being blocked by the LC.

becomes available. Similarly, a read operation will block if the channel is empty, forcing the component to wait until data arrives. We set up a multi-component messaging test case where a high criticality component sends data to a low criticality component through an intermediary. Since all components rely on blocking send operations, if the low criticality component stops reading, the intermediary—and consequently, the high criticality component—will be blocked when channels become full.

To address this blocking problem at the composition level, we applied a composition technique known as interposition. Interposition introduces an intermediary with non-blocking send operations between the middle and low criticality components, allowing asynchronous messaging. This modification allowed high criticality messages to bypass potential blocks created by the operations of low criticality component. Figure **??** illustrates how interposition resolves the blocking issue.

We measured end-to-end latency across components for both the default setup and a modified setup using an Interposition Solution. While this adjustment increased latency from 2450 to 3500 CPU cycles, it ensured that high criticality communications remained unimpeded, even if the low criticality component stalled.

Discussion. Applying interposition is slightly more complex compared to duplication, but it remains straightforward, as it requires no modifications to existing source code. In addition to adding the component's specification, a few lines of C code are needed in the newly created intermediary component. After introducing this component, the end-to-end latency across components increases by approximately 1000 CPU cycles. However, this trade-off is necessary to prevent unbounded latency for the high criticality component, ensuring predictable system performance.

DoS attack warnings and solution. A fundamental challenge in mixed criticality systems is the risk of low criticality components monopolizing shared resources, potentially leading to resource exhaustion. This monopolization could prevent high criticality components from accessing essential resources, undermining the system's reliability and responsiveness. This vulnerability can arise with any virtual resource where shared access could allow overuse or monopolization by less critical components.

As an example, consider a channel virtual resource that can be dynamically allocated and accessed by both high- and

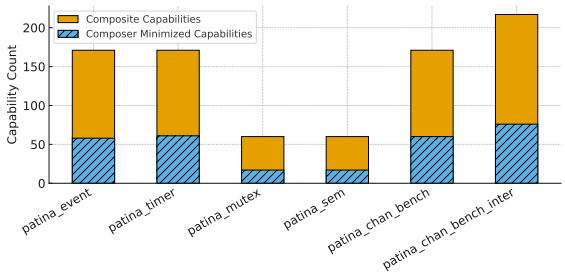


Fig. 11: Reduction in capability dependencies across Patina configurations before and after Composer's analysis.

low criticality components. If the low criticality component exhausts all available channel resources, the high criticality component will be unable to allocate any additional resources from the channel. This can prevent the high criticality component from performing its critical tasks, potentially leading to system performance failure to meet critical deadlines, compromising overall system reliability.

We applied again the duplication method to address this issue. By modifying the system specification to include a duplicated channel manager component, we ensured that each criticality level accesses its own resources independently, as shown in Figure **??**. This adjustment prevents low criticality components from monopolizing resources needed by high criticality components, mitigating DoS vulnerabilities.

Discussion. There are several general methods to prevent DoS vulnerabilities, and composition techniques like duplication may not always be the optimal solution. However, it enables DoS mitigation at the system configuration level without requiring modifications to source code. This flexibility is valuable in safety-critical systems where maintaining code stability is critical.

E. Capability Minimization

A key benefit of Composer is its ability to identify and eliminate unnecessary synchronous invocation capabilities between components. As described in §**??**, Composer combines a binary analysis to understand which functions can be called for each component entry point, with an inter-component analysis to understand which entry points are required in a component. Given this, it knows which functions cannot be called from which components, thus enabling it to soundly and completely minimize the number of PPC functional capabilities.

To assess this analysis's capabilities, we created a capability system with all Patina components (and core system components they depend on) with different Patina test components. Figure **??** shows the number of capabilities required by each configuration before and after Composer 's analysis. Composer removes roughly two-thirds of the synchronous invocation capabilities across all evaluated cases.

Discussion. These results demonstrate that Composer's static analysis significantly shrinks the capability footprint of a component-based system without requiring any modifications to application code. By eliminating unused PPC capabilities, Composer reduces the attack surface by more tightly adhering to the Principle of Least Privilege.

VIII. Related Work

Configurable systems. Common facilities for configuring the functionality of a given software are based around conditional compilation. For example, the Linux configuration system is famously complex [**?**], [**?**], and includes features relevant for real-time systems including scheduling policies and preemption modes. Even minimal systems such as FreeRTOS enable significant configuration, for example to enable memory protection facilities [**?**]. Similarly, unikernels [**?**], [**?**] allow developers to customize the OS image by selecting only the required library-kernels and drivers at build time. While this configuration is invaluable, it focuses only on enabling (or not) existing features that are often intertwined in the source code. This configuration focuses on feature selection rather than architectural correctness or safety reasoning. In contrast, component-based systems are based on the *separation of concerns* of various system features in different components. While this enables expressive definition of complex component graphs, it is difficult to reason about the composition. Composer enables build-time reasoning about component dependencies, isolation, and mixed-criticality constraints, ensuring safe and analyzable system composition.

Component-based systems. These systems emphasize modularity and isolation but rely on manual composition rather than declarative specification and analysis. In these systems, components are structured as isolated protection domains, but the relationships among them are implicitly defined in code or build scripts. L4Re [**?**] provides a C++ framework and Lua-based initialization model for constructing user-level services on top of the L4 microkernel. Tock OS [**?**], [**?**] structures embedded software into capsules, memory-safe Rust components statically linked into the kernel through traits and generics. Patina-seL4 [**?**] builds a bespoke component-based RTOS on top of seL4 and Composite. Hongmeng (HarmonyOS) [**?**] introduces a configurable composition model for microkernel-based systems, enabling flexible isolation and coalescing of OS services for performance. While Composer is implemented in Composite, its ideas could be ported into each of these systems, providing a high-level declarative view of the component graph, resources, and an analysis over component interactions.

Component-based systems with architecture specifications. These systems are the most similar to Composer and define not only the components themselves but also their interfaces, dependencies, and connections in a declarative manner. CAmkES allows developers to describe components, interfaces, and connectors using an Architecture Description Language (ADL), and CAmkES automatically generates the glue code, initialization logic, and capability mappings required for deployment [**?**]. It focuses on how seL4 kernel resources can be orchestrated to create a functional system. Further, it provides [**?**] a proof of confidentiality for the resulting system. Composer also orchestrates kernel resource allocation to create executable components, but also optimizes component resource allocations using system and binary analysis, provides abstractions to support component-definition of resources, and analysis to understand mixed-criticality interference.

AUTOSAR Classic standardizes the architecture of automotive embedded systems using similar specification ideas. System composition is expressed in AUTOSAR XML (ARXML) files that describe software components, ports, and communication paths [**?**]. To support dynamic behavior such as runtime resource allocation, a separate framework—the AUTOSAR Adaptive—must be used instead of the Classic platform [**?**]. In contrast, Composer enables dynamic behavior while preserving the invariants established during static composition.

ROS provides a modular, peer-to-peer framework that enables distributed robotic systems to integrate heterogeneous software components through message-based communication and language-neutral interfaces [**?**] . Based on ROS 2 [**?**], the real-time research community has developed end-to-end latency analysis [**?**]. Composer focuses on the static specification of component structure, and analysis to understand how multiple, independent, mixed-criticality components can interfere with each other. Whereas ROS focuses on composing *an application*, we focus on CBOSes that enable multiple independent applications to predictably execute. Research in ROS for end-to-end response time are complementary to Composer, and could be adapted given strong assumptions on component execution times.

IX. Conclusions

This paper presents Composer, a system for the configuration, linking, and loading of CBOSes. It is designed to enable analysis of component relations, dependencies, and interference in low-level systems in which abstractions, mechanisms, and policies are themselves implemented by components. Specifically, we orchestrate the specification, sharing, access rights, and allocation of *virtual resources* that are concretely defined by components. This lifts the representation of these resources from the code into the specification, and enables Composer to analyze the non-functional interactions between components. Composer analyses enable the right-sized static allocation and coordination of these resources between the resource provider and client. It also makes the dependencies between computations of various criticalities a first-class concern, enabling the system to identify specifications in which misbehaving low-criticality code can inhibit high-criticality progress, and where low-criticality code can cause undue latency through interference via shared system components. We provide a number of component graph translations to remove these issues, and demonstrate that they don't prohibitively increase latency. Finally, Composer integrates binary and inter-component analyses to optimize memory for stacks, and to tightly constrain communication access rights. We demonstrate its effectiveness in the Composite component-based OS. We believe that Composer provides a strong foundation for understanding, analysing, and adapting non-functional concerns in component-based systems.